Boosting Algorithms Explained: A Comparative Study of AdaBoost and Gradient Boosting Using the Titanic Dataset

Github: https://github.com/Ganesh-02-2003/Assignment/tree/main

Introduction

Machine learning models tend to analyse complex datasets and help in prediction. Several conventional algorithms assume the existence of a single model to learn the pattern behind the data. However, most of these models do not do so whenever there is noise in the data, or when there is a non-linear relationship in the data. To improve predictive performance, ensemble learning techniques combine multiple models to address this limitation.

Boosting is an ensemble strategy that creates models sequentially, with each new model learning from the errors made by earlier models. Boosting algorithms construct highly exact classifiers by sequentially correcting the preceding model.

There are two popular algorithms for boosting known as Ada Boost and Gradient Boosting. Both algorithms use sequential learning to achieve their respective goals; nevertheless, they do differ on how they change model errors and update model weights.

In this tutorial, you will learn about boosting algorithms via a comparison of AdaBoost and Gradient Boosting on the Titanic passenger dataset. The aim is to show how these algorithms give better prediction performance and how they behave as the number of estimators increases. The ability of boosting models to perform well on a wide variety of datasets is certainly well-known in data science.

Ensemble Learning and Boosting

The process of combining multiple models is known as ensemble learning. Ensemble methods take multiple models into consideration. We use multiple models and combine their predictions to make the final prediction.

Boosting, a type of ensemble learning, has models that are trained one after the other. With every successive model, an attempt is made to correct the mistakes of the earlier model. This process occurs repetitively to form a powerful prediction model from a collection of weak models.

In boosting algorithms how weak learners usually refers to a simple model such as shallow decision trees. Even though the prediction of each model is not strong, combining their predictions yields a strong classifier.

AdaBoost and Gradient Boosting come under the category of popular boosting algorithms. The two methods build models sequentially but differ in the way they weight misclassified examples and update the model during training.

AdaBoost Algorithm

AdaBoost, an acronym for Adaptive Boosting, was developed by Freund and Schapire (1997). Observations in the training data are weighted by the algorithm. First, all observation is equally important. After the first model of the algorithm is trained, it labels which observations are misclassified.

When an observation is misclassified, it is assigned a larger weight so that the next model places more importance on it. The classifier is improved through several rounds of this process Every model in AdaBoost contributes to the final decision where the weight it gets is proportional to how well it performs. The final ensemble is influenced more by models that do better.

The adaptive weighting method allows AdaBoost to learn from difficult observations, so the predictive accuracy of the ensemble increases.

Gradient Boosting Algorithm

Gradient boosting is another boosting technique which sequentially improves predictions of a model. Gradient Boosting instead optimises a loss function by gradient descent rather than observation weights.

The algorithm initiates training a simple model for predicting the target variable. After creation of first modal, the algorithm calculates the residual errors between predicted values and actual values. A new model is then trained to forecast these leftover errors.

The algorithm continuously reduces prediction errors by modelling the residuals. Each new model rectifies the errors committed by the earlier ones.

Gradient Boosting is very flexible as it can optimize many distinct kinds of loss functions. The algorithm can generalize well to various machine learning tasks due to this capability.

Dataset Description

The Titanic data has information about the passengers of the RMS Titanic. During its maiden voyage in April 1912, the ship hit an iceberg and sank. Passenger characteristics contained in the dataset could be used to analyse the survival.

Several variables were selected for this tutorial, including:

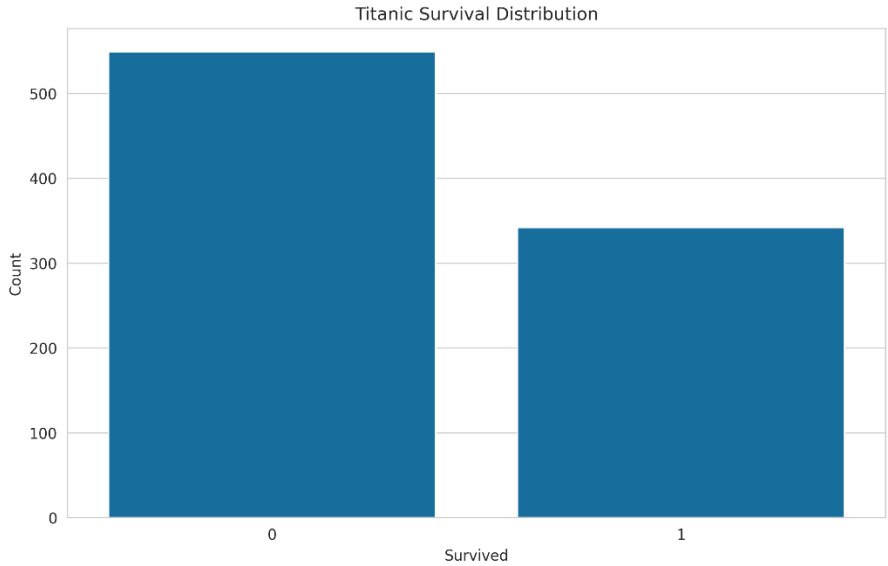
- Passenger class
- Sex
- Age
- Number of siblings or spouses aboard
- Number of parents or children aboard
- Ticket fare
- Port of embarkation

The target variable in the dataset is **survived**, which shows whether a passenger survived the disaster.

Prior to boosting models, the dataset should be pre-processed. The median age of the passengers was used to replace the missing age variables. The most common category was used to replace missing values. One-hot encoding was applied to convert categorical variables such as gender and embarkation port into numerical values.

Exploratory Data Analysis

Exploratory data analysis helps provide insight into the structure of the dataset before building predictive models.



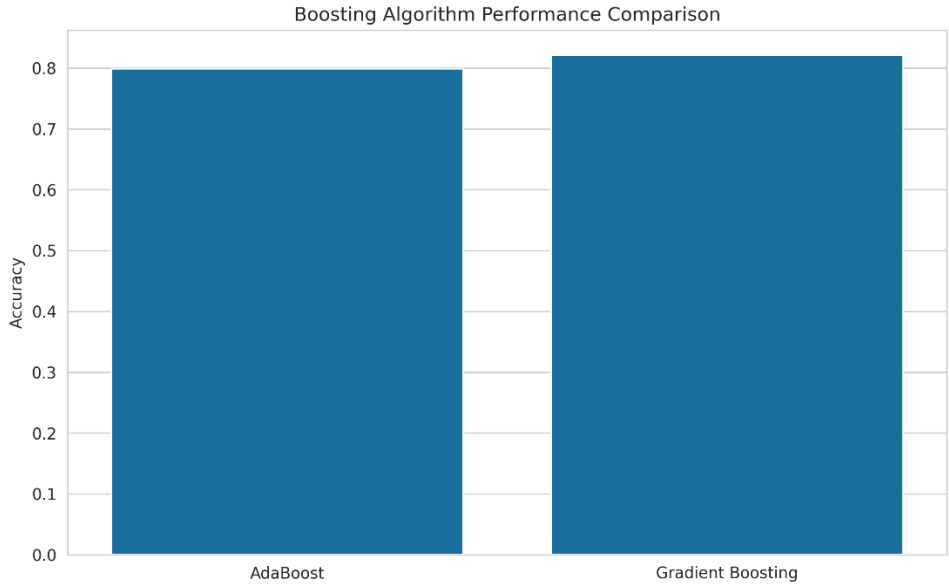
The graph illustrates that the total number of casualties exceeds the number of victims who survived. This imbalance has its roots in the outcome of the disaster itself, showing that surviving is hard to predict.

Looking at the dataset can find other variables that may affect survival. For instance, earlier study found that gender and passenger class significantly increased the chances of survival in the Titanic evacuation.

Model Training and Comparison

Two boosting models were trained for this tutorial: AdaBoost and Gradient Boosting. Both models used decision trees as base learners and were trained using the same training and testing dataset split.

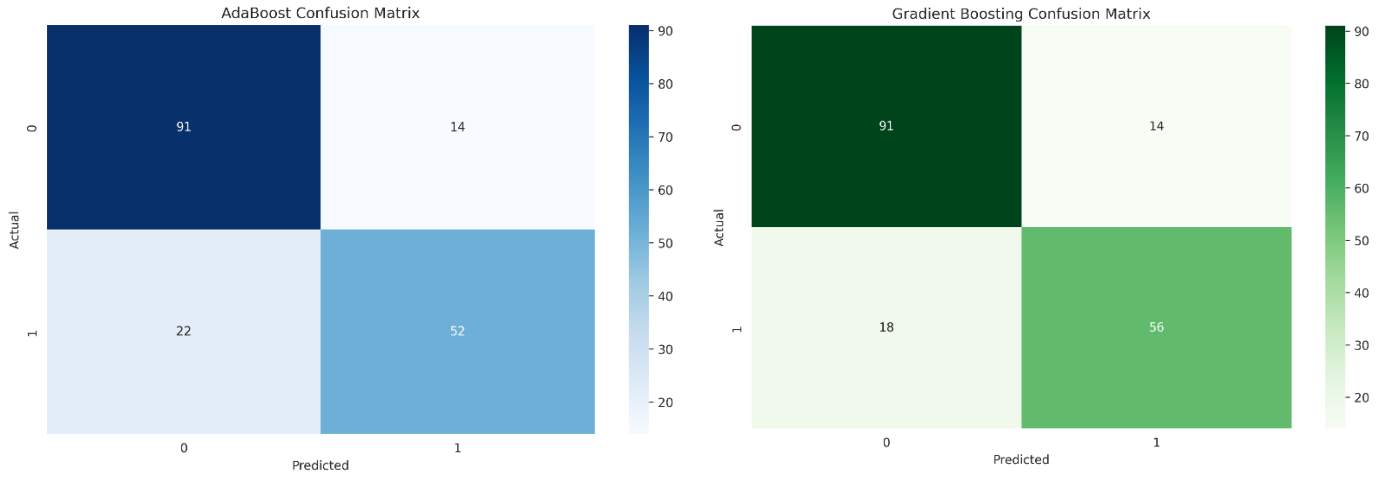
After training the models, their predictive performance was evaluated using accuracy as the primary metric.



The comparison shows that both boosting methods achieve strong predictive performance on the Titanic dataset. Although the difference between the models may be small, Gradient Boosting often proves slightly better performance due to its ability to optimise prediction errors more precisely.

Model Evaluation

To further analyse model performance, confusion matrices were generated for both algorithms.



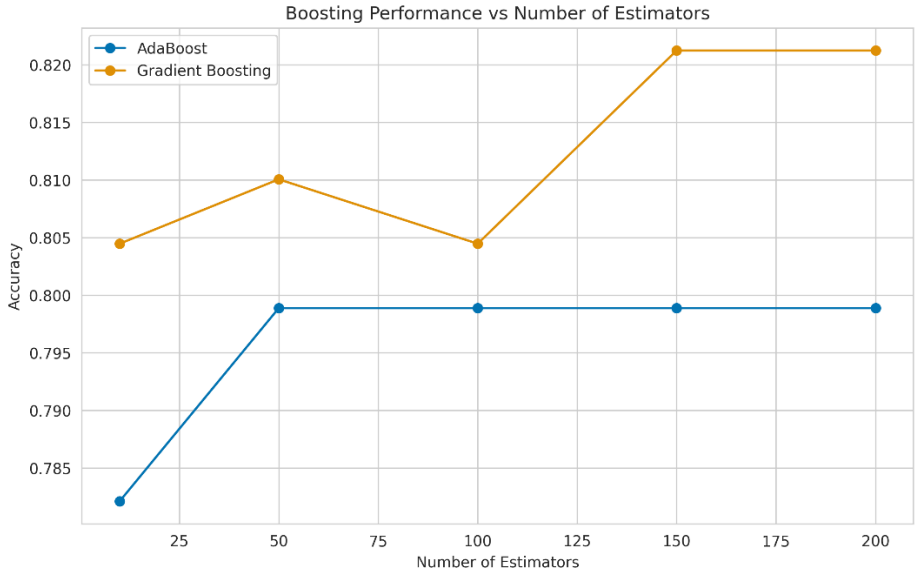
The number of correctly and incorrectly classified observations are summarised in the confusion matrices. True positives mean correctly predicted survivor. True negative means correctly predicted non-survivor.

Both models show a strong classification capacity, properly classifying a considerable proportion of passengers in both categories.

Effect of Number of Estimators

The number of estimators is an important parameter of boosting algorithms that decides weak learners in the ensemble.

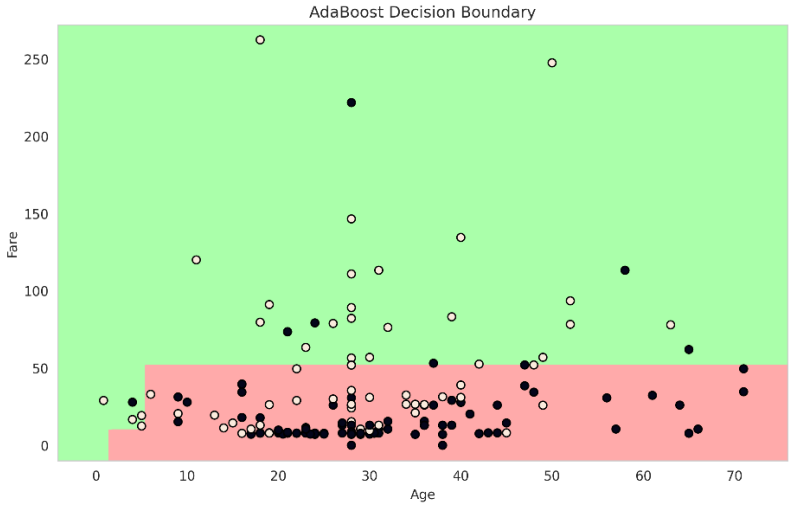
With a larger number of estimators, the model can learn more complex patterns in the data. Nonetheless, using extremely large ensembles may raise the cost of computation and increase the risk of overfitting.

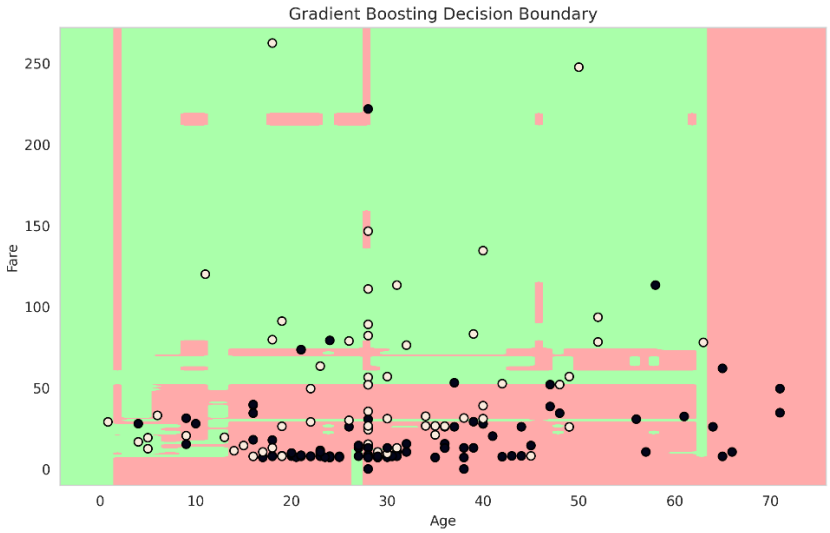


The plot proves that accuracy generally improves as more estimators are added, although the improvement eventually stabilises. This behaviour illustrates how boosting gradually refines the model by combining many weak learners.

Decision Boundary Visualisation

To further illustrate the behaviour of boosting algorithms, a decision boundary visualisation was generated using two features: age and fare.





These visualisations highlight how boosting algorithms partition the feature space to classify observations. The boundaries produced by Gradient Boosting appear smoother because the algorithm optimises prediction errors iteratively using gradient descent.

This visual comparison provides an intuitive understanding of how boosting models learn decision boundaries from data.

Discussion

The use of boosting algorithms for classification tasks is seen in AdaBoost vs Gradient Boosting Comparison. Both model's results show that boosting methods could effectively combine weak learners towards strong prediction on the Titanic dataset.

Both algorithms are based on similar principles, however, unlike AdaBoost, Gradient Boosting is more flexible as it uses gradient descent to optimise errors when making predictions. This enables it to adapt better with complex data patterns.

The visualizations you will create in this tutorial help illustrate how boosting algorithms learn sequentially to improve the performance of models.

Limitations

Despite their robust performance, boosting algorithms have several limitations. Boosting models can be sensitive to noisy data because the sequential training process may repeatedly focus on difficult observations.

In addition, boosting models require careful tuning of hyperparameters such as the number of estimators and learning rate. Poor parameter choices may lead to overfitting or reduced performance.

Finally, while boosting models often achieve high accuracy, their sequential structure can make them computationally expensive when applied to large datasets.

Future Work

Future work could extend this analysis by exploring more advanced boosting algorithms such as XGBoost or LightGBM. These modern gradient boosting frameworks often achieve even higher predictive performance and are widely used in real-world machine learning applications.

Conclusion

AdaBoost and Gradient Boosting Algorithms on the Titanic Dataset was what we covered in this tutorial. The analysis showed that the prediction of boosting algorithms combines weak learners sequentially.

The two models showed high classification accuracy and were able to find patterns. Learning curves and plots of decision boundary showed how boosting approaches improve their predictions which offer us more useful insights.

The results show how boosting technique is effective in machine learning in addition to how ensemble methods enhance model performances.

References

Freund, Y. and Schapire, R. (1997) A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. Journal of Computer and System Sciences, 55(1), pp. 119–139.

Friedman, J. (2001) Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), pp. 1189–1232.

Breiman, L. (2001) Random Forests. Machine Learning, 45(1), pp. 5–32.

Pedregosa, F. et al. (2011) Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

Géron, A. (2022) Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow. O’Reilly Media.